```
a = %w{ ant bee cat dog elk }
```

Ruby hashes are similar to arrays. A hash literal uses braces rather than square brackets. The literal must supply two objects for every entry: one for the key, the other for the value. For example, you may want to map musical instruments to their orchestral sections:

```
inst_section = {
  :cello     => 'string',
  :clarinet  => 'woodwind',
  :drum      => 'percussion',
  :oboe      => 'woodwind',
  :trumpet   => 'brass',
  :violin    => 'string'
}
```

The thing to the left of the `=>` is the key, and that on the right is the corresponding value. Keys in a particular hash must be unique; if you have two entries for `:drum`, the last one will *win*. The keys and values in a hash can be arbitrary objects: you can have hashes in which the values are arrays, other hashes, and so on. In Rails, hashes typically use symbols as keys. Many Rails hashes have been subtly modified so that you can use either a string or a symbol interchangeably as a key when inserting and looking up values.

The use of symbols as hash keys is so commonplace that Ruby has a special syntax for it, saving both keystrokes and eyestrain:

```
inst_section = {
  cello:    'string',
  clarinet: 'woodwind',
  drum:     'percussion',
  oboe:     'woodwind',
  trumpet:  'brass',
  violin:   'string'
}
```

Doesn’t that look much better?

Feel free to use whichever syntax you like. You can even intermix usages in a single expression. Obviously, you’ll need to use the arrow syntax whenever the